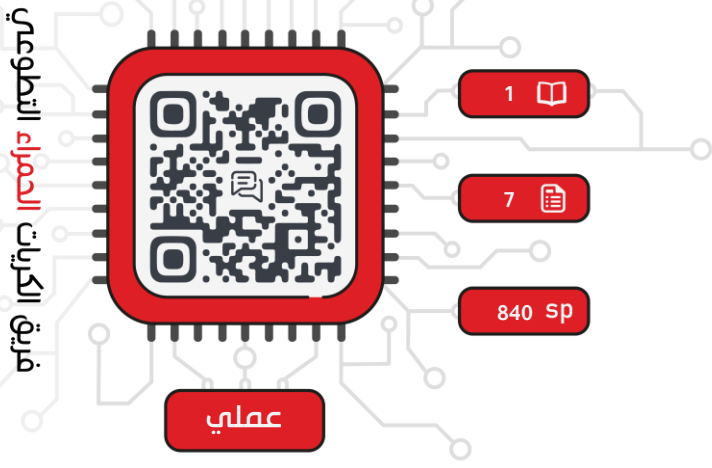


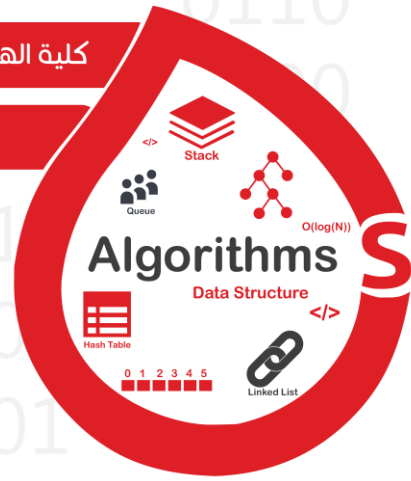


كلية الهندسة المعلوماتية

السنة الثانية

أنواع التعقيد

عملي مشترك



RB Informatcs; 21/10/2025

خوارزميات 1

الخوارزمية

الخوارزمية هي مجموعة من الخطوات المتسلسلة و المنطقية المنتهية لحل مشكلة ما.

الطرق المستخدمة في كتابة الخوارزمية

1. المخطط التدفقي Flow Chart

عبارة عن مجموعة من الأشكال الهندسيّة التي نستخدمها لتمثيل تعليمات معيّنة.

2. اللّغة النصيّة Pseudo Code

طريقة تعرض حل مسألة معينة بلغة بسيطة غير رسميّة، رموز البرمجة الأساسيّة مع بعض الكلمات (العربية / الانكليزية) للشرح.. أي؛ هي مسوّد للمبرمجين لفهم الحل قبل كتابة الكود الحقيقي. و لمعرفة الفرق لدينا المثال الآتي:

C++ Code	Pseudo Code
<pre>cin >> t; for(int i = 0; i <= t; i++) if(i % 2 == 1) cout << i << endl;</pre>	<pre>-Start: read t -For i from 0 to t Do -If i mod 2 equals 1 -THEN print i -End if -End for -END</pre>

أنواع التعقيد

التعقيد الذاكري/المكاني Space Complexity

حساب الحجم الذي يستهلكه البرنامج من الذاكرة و يمكن حسابه بسهولة عن طريق حساب حجم الـ Data Structure المستخدمة.

double l=3.14	8 byte
int Array[5]	4 byte × 5 = 20 Byte

سندرس الآن مثلاً عن الأخطاء التي تحدث بسبب استهلاك الذاكرة Stack Over Flow

خطأ برمجي يحصل عندما تمتلئ الـ stack و هي الذاكرة المخصصة للـ function calls و المتغيرات المحلية، و تتجاوز حدودها غالباً بسبب استخدام recursion (دالة تستدعي نفسها دون توقف) بطريقة خاطئة، أو Array كبيرة.

مثال:

```
fun() {
  print("Hi");
}
```

تذكرة:

العودية دالة تستدعي نفسها لحل مشكلة ما عن طريق تقسيمها لمشاكل أصغر وصولاً للحالة الأساسية Base Case، و هي أصغر جزء من المشكلة و عندها يتوقف استدعاء الدالة.

و الآن سنعرض عليكم بعض الأكواد لاختباركم ، عليكم معرفة الخطأ في الكود الذي يسبب infinite recursion:

الكود الأول	الكود الثاني
<pre>count++; fun () { count++; if (count<5) { print ("Hi"); fun (); } count++; }</pre>	<pre>count++; fun () { count++; if (count<5) { print ("Hi"); } fun (); }</pre>

و الآن سنعرف معاً و نكتشف السبب الذي أدّى إلى الـ Stack Over Flow:

■ الكود الأول:

المشكلة تكون استدعاء التابع (fun) يكون خارج الشرط if أي يتم استدعاء التابع إن تحقق الشرط أو لم يتحقق.

■ الكود الثاني:

العداد count تبقى قيمته صفر لأن الزيادة تكون بعد استدعاء التابع (fun) كل مرة التابع يستدعي نفسه مجدداً قبل الوصول الى تعليمة الزيادة.

بعض الحلول الممكنة و الصحيحة لمشكلتنا:

```
count=0;
fun () {
    if (count>=5)
        return;
    count++;
    print ("Hi");
    fun ();
}
```

و أيضاً:

```
count=0;
fun () {
    count++;
    if (count<5) {
        print ("Hi");
        fun ();
    }
}
```



التعقيد الزمني Time complexity

حساب الزمن الذي سيستغرقه البرنامج ليتم تنفيذ، و سنركز على هذا النوع من التعقيد في دراستنا، حيث:
زمن التنفيذ = عدد العمليات $\times$ زمن كل عملية.

يوجد لدينا نوعين من العمليات

1. العمليات المكلفة للوقت:

- العمليات الحسابية (الجمع و الطرح)
- الحلقات بأشكالها for, do while, while (و نأخذ كل العمليات بداخلها بعين الاعتبار لأنها تتكرر)
- المقارنات المنطقية if, else

2. العمليات غير المكلفة للوقت

- عملية الاسناد
- عملية زيادة عدّد الحلقات for(.....i++)

تحديد الخوارزمية الأفضل

يكون ذلك بالمقارنة بين الخوارزميات وفق معيارين:

1. أداء الخوارزمية Algorithms Performance

أي التعقيد الزمني و المكاني (الذاكري).

2. قابلية تطوير الخوارزمية Algorithms Scalability

امكانية التعديل عليها و قدرتها على التعامل مع مدخلات كبيرة دون زيادة كبيرة في التعقيد.





تمارين

التمرين الأول: أوجد العامل (Factorial).

طريقة التكرار:

```
fact = 1;
for(int i = n ; i >= 2; i--){
    fact *= i;
}
System.out.print(fact);
```

طريقة العودية:

```
int fact(int n){
    if(n == 1 || n == 0)
        return 1;
    return n*fact(n-1);
}
```

التمرين الثاني: إيجاد آخر عدد فردي في مصفوفة.

طريقة التكرار:

المدخل: مصفوفة A بطول N
العملية:

- أنشئ متغير lastOdd = لا شيء
- كرر من $i = 0$ إلى $N - 1$
- إذا $A[i] \% 2 \neq 0$
- lastOdd = A[i]
- اطبع lastOdd

طريقة العودية:

الوظيفة: FindLastOdd(A, i, lastOdd)
 الشرط النهائي: إذا $i \leq \text{طول } A$ ، اطبع lastOdd و توقف العملية:

- إذا $\text{lastOdd} = A[i]$, $A[i] \% 2 \neq 0$
 - استدع FindLastOdd(A, i+1, lastOdd)
 الاستدعاء الأول: (لا شيء, 0, A) FindLastOdd

التمرين الثالث: الفرق بين أكبر عدد بالمصفوفة و أصغر عدد (min, max).**طريقة التكرار:**

المدخل: مصفوفة A بطول N
 العملية:

- أنشئ $\text{min} = A[0]$, $\text{max} = A[0]$
 - كرر من $i = 1$ إلى $N - 1$
 - إذا $\text{max} < A[i]$, $\text{max} = A[i]$
 - إذا $\text{min} > A[i]$, $\text{min} = A[i]$
 - اطبع الفرق: $\text{max} - \text{min}$

طريقة العودية:

الوظيفة: FindMinMax(A, i, min, max)
 الشرط النهائي: إذا $i \leq \text{طول } A$ ، اطبع الفرق $\text{max} - \text{min}$ و توقف العملية:

- إذا $\text{max} < A[i]$, $\text{max} = A[i]$
 - إذا $\text{min} > A[i]$, $\text{min} = A[i]$
 - استدع FindMinMax(A, i+1, min, max)
 الاستدعاء الأول: FindMinMax(A, 1, A[0], A[0])

التمرين الرابع: العناصر الفريدة (غير المكررة) في المصفوفة.

طريقة التكرار:

المدخل: مصفوفة A بطول N

العملية:

- أنشئ مصفوفة فارغة اسمها Unique

- كرر من $i = 0$ إلى $N - 1$

- أنشئ عداد $count = 0$

- كرر من $j = 0$ إلى $N - 1$

- إذا $A[i] == A[j]$ زد count

- إذا $count == 1$ أضف $A[i]$ إلى Unique

- اطبع عناصر Unique

طريقة العودية:

الوظيفة: FindUnique (A, i, Unique)

الشرط النهائي: إذا $i \leq \text{طول } A$ توقف

العملية:

- أنشئ عداد $count = 0$

- كرر من $j = 0$ إلى $A - 1$

- إذا $A[i] == A[j]$ زد count

- إذا $count == 1$ أضف $A[i]$ إلى Unique

- استدع FindUnique (A, i+1, Unique)

الاستدعاء الأول FindUnique (A, 1, [])



انتهت المحاضرة

نلتقاكم في المحاضرة القادمة